目录

图论

最小生成树 - Prim 算法 (堆优化版)

算法说明

用途：求无向图的最小生成树 (MST) 的权值之和。

时间复杂度：O(ElogE) 或 O(ElogV)。

适用场景：适合稠密图，对于稀疏图，Kruskal 算法通常代码更短，但堆优化的 Prim 性能也十分优秀。

最小生成树 prim.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 5050, M = 2e5 + 10;
const int INF = 0x3f3f3f3f;

// 链式前向星存图
struct E {
    int v, w, x;
} e[M * 2]; // 无向图记得开两倍空间

int n, m, h[N], cnte;
int dis[N];     // 存储点到当前生成树集合的最小距离
bool vis[N];    // 标记点是否已加入生成树
int res = 0;    // 最小生成树权值和
int cnt = 0;    // 已加入生成树的点数量

void adde(int u, int v, int w) {
    e[++cnte] = E{ v, w, h[u] };
    h[u] = cnte;
}

struct S {
    int u, d;
    // 优先队列小根堆，距离小的优先
    bool operator<(const S& x) const {
        return d > x.d;
    }
};

priority_queue<S> q;

// 返回 true 表示成功生成 MST，false 表示图不连通
bool Prim() {
    // 初始化
    memset(dis, 0x3f, sizeof(dis));
    memset(vis, 0, sizeof(vis));
    res = 0;
    cnt = 0;

    // 从 1 号点开始
    dis[1] = 0;
    q.push({ 1, 0 });

    while(!q.empty()) {
        // 优化：如果已经选了 n 个点，说明生成树构建完成
        if(cnt >= n) break;

        int u = q.top().u;
        int d = q.top().d;
        q.pop();

        if(vis[u]) continue;
        vis[u] = 1;
        res += d;
        cnt++;

        for(int i = h[u]; i; i = e[i].x) {
            int v = e[i].v;
            int w = e[i].w;
            // 如果通过 u 到达 v 的距离比 v 当前记录的距离更短
            if(w < dis[v]) {
                dis[v] = w;
                q.push({ v, w });
            }
        }
    }

    // 判断是否所有点都在生成树内
    return cnt == n;
}

/* 使用示例：
 cin >> n >> m;
 for(int i=1; i<=m; i++){
     int u,v,w; cin>>u>>v>>w;
     adde(u,v,w); adde(v,u,w);
 }
 if(Prim()) cout << res << endl;
 else cout << "impossible" << endl;
*/
```